Java OOPs Interview Guide

Comprehensive Explanations & Detailed Code Examples

Foundations of OOP

1. What is OOP?

OOP (Object-Oriented Programming) is a programming paradigm where we design and build applications utilizing classes and objects rather than relying strictly on functions and logic.

> ***Interview Answer:*** *OOP is a programming paradigm that organizes code using classes and objects. It significantly improves reusability, security, and the long-term maintainability of applications.*

2 & 3. Class and Object Example

A class is a blueprint, and an object is a real-world instance of that blueprint.

Class and Object Initialization

```
class Student {
    String name;
    int age;

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an object of the Student class
        Student s1 = new Student();
        s1.name = "Alice";
        s1.age = 22;
        s1.display(); // Output: Name: Alice, Age: 22
    }
}
```

Pillar 1: Encapsulation

5. What is Encapsulation?

Encapsulation is the mechanism of wrapping both data (variables) and code acting on the data (methods) together as a single cohesive unit. It is achieved by declaring variables as `private` and providing public `getter` and `setter` methods.

Complete Encapsulation Example

```
class Employee {
    private String name;   // Data hiding
    private double salary; // Data hiding

    // Getter for name
    public String getName() {
        return name;
    }
    // Setter for name
    public void setName(String name) {
        this.name = name;
    }

    // Getter for salary
    public double getSalary() {
        return salary;
    }
    // Setter for salary with validation logic
    public void setSalary(double salary) {
        if(salary > 0) {
            this.salary = salary;
        } else {
            System.out.println("Error: Salary must be positive.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.setName("John Doe");
        emp.setSalary(55000.50);

        System.out.println("Employee: " + emp.getName());
        System.out.println("Salary: $" + emp.getSalary());
    }
}
```

> ***Interview Answer:*** *Encapsulation is the process of hiding internal data using private variables and providing controlled access to that data through public getter and setter methods.*

Pillar 2: Inheritance

7 & 8. Types of Inheritance

Inheritance allows a new class to acquire properties and methods of an existing class. Java supports Single, Multilevel, and Hierarchical inheritance through classes.

1. Single Inheritance (One Parent, One Child)

```
class Animal {
    void eat() { System.out.println("Eating..."); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();  // Inherited from Animal
        d.bark(); // Own method
    }
}
```

2. Multilevel Inheritance (Chain of Inheritance)

```
class Animal {
    void eat() { System.out.println("Eating..."); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}
class Puppy extends Dog {
    void weep() { System.out.println("Weeping..."); }
}

public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();  // Inherited from Animal
        p.bark(); // Inherited from Dog
        p.weep(); // Own method
    }
}
```

3. Hierarchical Inheritance (One Parent, Multiple Children)

```
class Animal {
    void eat() { System.out.println("Eating..."); }
}
class Cat extends Animal {
    void meow() { System.out.println("Meowing..."); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}

public class Main {
    public static void main(String[] args) {
        Cat c = new Cat();
        c.meow();
        c.eat(); // Inherited from Animal

        Dog d = new Dog();
        d.bark();
        d.eat(); // Inherited from Animal
    }
}
```

Pillar 3: Polymorphism

11. Compile-Time Polymorphism (Method Overloading)

Method overloading happens when multiple methods in the same class share a name but have different parameter lists. It is resolved at compile-time.

Method Overloading Example

```
class MathOperations {
    // Method with 2 integer parameters
    int add(int a, int b) {
        return a + b;
    }
    // Method with 3 integer parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }
    // Method with 2 double parameters
    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperations math = new MathOperations();
        System.out.println(math.add(5, 10));       // Calls int, int
        System.out.println(math.add(5, 10, 15));   // Calls int, int, int
        System.out.println(math.add(2.5, 3.5));    // Calls double, double
    }
}
```

12. Runtime Polymorphism (Method Overriding)

Method overriding happens when a child class provides a specific implementation for a method already defined in its parent class. It is resolved dynamically at runtime.

Method Overriding Example

```
class Bank {
    int getInterestRate() {
        return 0;
    }
}
class SBI extends Bank {
    @Override
    int getInterestRate() {
        return 7; // Specific implementation for SBI
    }
}
class HDFC extends Bank {
    @Override
    int getInterestRate() {
        return 8; // Specific implementation for HDFC
    }
}

public class Main {
    public static void main(String[] args) {
        // Parent reference pointing to child objects
        Bank myBank1 = new SBI();
        Bank myBank2 = new HDFC();

        System.out.println("SBI Rate: " + myBank1.getInterestRate() + "%");
        System.out.println("HDFC Rate: " + myBank2.getInterestRate() + "%");
    }
}
```

Pillar 4: Abstraction

16. Abstract Class Example

An abstract class is declared with the `abstract` keyword and provides partial abstraction. It can have both abstract methods (no body) and concrete methods (with a body).

Abstract Class Implementation

```
abstract class Shape {
    // Abstract method (no implementation)
    abstract void draw();

    // Concrete method (shared logic)
    void displayArea() {
        System.out.println("Displaying shape details.");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle.");
    }
}

public class Main {
    public static void main(String[] args) {
        // Shape s = new Shape(); // Error: Cannot instantiate abstract class
        Shape myShape = new Circle();
        myShape.draw();       // Calls overridden child method
        myShape.displayArea(); // Calls inherited parent method
    }
}
```

17. Interface Example

An interface acts as a strict contract, ensuring complete abstraction (traditionally). A class implements an interface and must provide bodies for all its abstract methods.

Interface Implementation

```
interface Payment {
    // Abstract method by default
    void pay(double amount);
}

class CreditCardPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " using Credit Card.");
    }
}

class UPIPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " using UPI.");
    }
}

public class Main {
    public static void main(String[] args) {
        Payment p1 = new CreditCardPayment();
        Payment p2 = new UPIPayment();

        p1.pay(150.00);
        p2.pay(25.50);
    }
}
```